

01 QUERY STRUCTURE

Skeleton

```
SELECT [DISTINCT] expressions
FROM R [JOIN S ON condition]
[WHERE row_condition]
[GROUP BY keys]
[HAVING group_condition]
[ORDER BY expression [ASC|DESC]]
[LIMIT n];
```

Logical processing order

FROM/JOIN → WHERE → GROUP BY
→ HAVING → SELECT
→ ORDER BY → LIMIT

! Window ORDER BY controls the window calculation, not the final output order. Add an outer ORDER BY when display order matters.

02 NULL / AGGREGATION

x = NULL is wrong; use x IS NULL.

Comparisons with NULL yield UNKNOWN; WHERE keeps only TRUE. Arithmetic with NULL usually returns NULL.

Counts COUNT(*) counts output rows; COUNT(x) ignores NULL; COUNT(DISTINCT x) counts unique non-NULL values.

COND Every selected nonaggregate expression must be grouped in standard SQL. WHERE filters rows before grouping; HAVING filters groups after aggregation.

! After a LEFT JOIN, COUNT(*) still counts a NULL-extended row; COUNT(B.id) ignores it.

03 JOIN / SET / SUBQUERY

Example data A(id,name): (1,Ana),(2,Bo); B(id,tag): (1,DB),(3,AI). Match on id; results are (A.name,B.tag).

```
INNER: A INNER JOIN B ON A.id=B.id
CROSS: A CROSS JOIN B
LEFT:  A LEFT JOIN B ON A.id=B.id
RIGHT: A RIGHT JOIN B ON A.id=B.id
FULL:  A FULL OUTER JOIN B ON A.id=B.id
```

- INNER: (Ana,DB).
- CROSS: all 4 pairs: (Ana,DB), (Ana,AI), (Bo,DB), (Bo,AI).
- LEFT: (Ana,DB), (Bo,NULL).
- RIGHT: (Ana,DB), (NULL,AI).
- FULL OUTER: (Ana,DB), (Bo,NULL), (NULL,AI).

COND Bare JOIN defaults to INNER JOIN; MySQL has no direct FULL OUTER JOIN syntax.

ON vs WHERE

A LEFT JOIN B ON A.id=B.id AND B.tag='DB'

The ON filter keeps Bo with NULL; putting B.tag='DB' in WHERE removes Bo.

Join traps NATURAL JOIN matches every same-named column, which can join unrelated fields; USING(k) names shared keys explicitly. Anti-join: LEFT JOIN, then WHERE the right key IS NULL.

Set operations UNION / INTERSECT / EXCEPT require union-compatible columns. They remove duplicates by default; UNION ALL preserves duplicates.

Subqueries

```
WHERE x IN (SELECT ...)
WHERE EXISTS (SELECT 1 FROM S WHERE ...)
WHERE NOT EXISTS (...)
```

Correlated subqueries refer to outer-row aliases. For “for all”, test that there is no missing requirement (e.g. required set EXCEPT possessed set is empty).

! NOT IN with any NULL in its subquery can yield UNKNOWN; NOT EXISTS is safer for anti-joins.

04 DDL / DML

```
CREATE TABLE R (
  id INT PRIMARY KEY,
  x VARCHAR(20) NOT NULL,
  dept VARCHAR(20),
  CHECK (...),
  FOREIGN KEY (dept) REFERENCES D(dept)
);
INSERT INTO R(...) VALUES (...);
UPDATE R SET x=... WHERE ...;
DELETE FROM R WHERE ...;
```

COND Foreign-key actions include ON DELETE/UPDATE CASCADE, SET NULL, and RESTRICT/NO ACTION.

CTE

```
WITH X AS (SELECT ...)
SELECT ... FROM X;
```

05 RANKING / WINDOWS

For scores 100, 100, 90:

Function	100	100	90
ROW_NUMBER	1	2	3
RANK	1	1	3
DENSE_RANK	1	1	2

ROW_NUMBER is sequential and unique; ties need a deterministic tie-breaker. RANK leaves gaps after ties; DENSE_RANK does not.

```
RANK() OVER (
  PARTITION BY dept
  ORDER BY score DESC)
```

PARTITION BY restarts within each group; ORDER BY sets window order. Ranking on grouped results happens after grouping/aggregation.

Window form

```
window_fn(expr) OVER (
  [PARTITION BY p] [ORDER BY o]
  [ROWS frame])
```

Windows calculate per row without collapsing rows. Aggregate windows include SUM/AVG/COUNT/MIN/MAX; ranking functions are nonaggregate.

Frames

```
-- Running: first row through current
SUM(x) OVER (ORDER BY d ROWS BETWEEN
  UNBOUNDED PRECEDING AND CURRENT ROW)
-- Moving: previous 2 plus current
AVG(x) OVER (ORDER BY d ROWS BETWEEN
  2 PRECEDING AND CURRENT ROW)
-- Symmetric: previous, current, next
AVG(x) OVER (ORDER BY d ROWS BETWEEN
  1 PRECEDING AND 1 FOLLOWING)
```

! ROWS counts physical rows. RANGE groups ORDER BY peers, so ties can enlarge the frame.

06 RECURSIVE CTE

Use for transitive closure, hierarchies, and paths.

```
WITH RECURSIVE reach(a,b) AS (
  -- Anchor: base edges
  SELECT a,b FROM edge
  UNION
  -- Recursive term extends prior rows
  SELECT r.a,e.b FROM reach r
  JOIN edge e ON r.b=e.a
)
SELECT * FROM reach;
```

The anchor seeds rows; the recursive term refers to the CTE and repeatedly extends it.

COND UNION removes duplicates and can help closure/cycle control. UNION ALL avoids deduplication and needs a safe termination strategy.

07 PROCEDURES / TRIGGERS

MySQL stored procedure

```
DELIMITER //
CREATE PROCEDURE p(IN p_id INT, OUT p_cnt INT)
BEGIN
  DECLARE v INT DEFAULT 0;
  SELECT COUNT(*) INTO v FROM T WHERE id=
    p_id;
  SET p_cnt=v;
END//
DELIMITER ;
CALL p(7,@n); SELECT @n;
```

IN receives a caller value; OUT returns one. Declare locals/conditions/handlers before executable statements. SELECT ... INTO stores a scalar result. DELIMITER is a client command so internal semicolons stay in the procedure body.

Triggers Fire on INSERT, UPDATE, or DELETE. BEFORE validates or changes NEW; AFTER runs after success, e.g. for audit work.

```
CREATE TRIGGER trg BEFORE UPDATE ON T
FOR EACH ROW
BEGIN
  IF NEW.score < 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT='bad score';
  END IF;
END;
```

Row images: INSERT uses NEW; UPDATE uses OLD and NEW; DELETE uses OLD. SIGNAL raises a user error; a trigger error aborts the invoking statement.

08 PIVOT / CUBE / ROLLUP

Conditional aggregation Rows become grouping keys; CASE values become output columns.

General PIVOT pattern

```
SELECT row_dimension,
  AGG(CASE WHEN pivot_column = 'value1'
    THEN measure ELSE 0 END) AS value1,
  AGG(CASE WHEN pivot_column = 'value2'
    THEN measure ELSE 0 END) AS value2
FROM table_name
GROUP BY row_dimension;
```

💡 PIVOT = GROUP BY rows + CASE WHEN columns + AGG cells.

Example

```
SELECT location,
  SUM(CASE WHEN type='THEFT' THEN 1 ELSE 0
    END) theft,
  SUM(CASE WHEN type='BATTERY' THEN 1 ELSE 0
    END) battery
FROM crime GROUP BY location;
```

For amounts, replace 1 with amount. COUNT(CASE WHEN condition THEN 1 END) also counts matches because COUNT ignores NULL.

Grouping sets For dimensions (a,b,c):

- CUBE(a,b,c): every subset, $2^3 = 8$ sets.
- ROLLUP(a,b,c): prefix hierarchy (abc, ab, a, empty), detail to subtotals to total.
- GROUPING(a)=1 when a is omitted in a subtotal/total; use it to distinguish generated NULL from data NULL.

MySQL supports ROLLUP. CUBE/general grouping-set syntax is commonly shown with PostgreSQL in course material.

09 LAST-MINUTE TRAPS

- Window functions preserve row count; GROUP BY collapses rows.
- RANK has gaps; DENSE_RANK has none; ROW_NUMBER is unique.
- ROWS BETWEEN n PRECEDING AND CURRENT ROW covers at most $n + 1$ physical rows at an interior row.
- Recursive CTE: anchor first; recursive term must reach a fixpoint or stop.
- Trigger row images: INSERT=NEW, DELETE=OLD, UPDATE=both.
- GROUPING() separates subtotal NULL from a real NULL value.

10 DBMS / RELATIONAL MODEL

Why a DBMS? Controls redundancy/inconsistency, difficult access, isolation, buried integrity rules, partial updates, concurrency, and authorization.

Abstraction Physical: how data is stored. Logical: which data and relationships exist. View: a user-specific representation.

COND Physical data independence means changing physical storage without changing the logical schema or applications.

Schema / instance Schema is structure; instance is a snapshot of tuples. A relation is a set of tuples (order is irrelevant); a domain is a set of allowed atomic values. NULL denotes unknown/missing.

Keys Superkey uniquely identifies tuples; candidate key is a minimal superkey; primary key is the selected candidate key. A foreign key references a key in another relation; NULL is allowed only if the column permits it.

11 RELATIONAL ALGEBRA

Rows	$\sigma_{condition}(R)$
Columns	$\pi_{columns}(R)$
All pairs	$R \times S$
Matched pairs	$R \bowtie_{condition} S$
Either set	$R \cup S$
R but not S	$R - S$
Rename	$\rho_X(R)$
Assign result	$X \leftarrow E$

RA to SQL σ = WHERE; π = SELECT DISTINCT; $\bowtie$ = JOIN ... ON; $\cup$ = UNION; $-$ = EXCEPT/anti-join; ρ = alias AS.

! RA projection removes duplicates. SQL SELECT preserves them unless DISTINCT is requested.

12 PHP / MYSQL SYNTAX

PHP patterns

```
$x // variable
$s = $a . $b; // concatenate strings
foreach ($xs as $x) { ... }
$row["column"] // associative-array
    field
$_GET["x"]; $_POST["x"];
$conn = mysqli_connect(...);
$r = mysqli_query($conn, $sql);
$rows = mysqli_fetch_all($r, MYSQLI_ASSOC);
if ($_SERVER["REQUEST_METHOD"] === "POST")
{ ... }
```

HTML form / dropdown

```
<form method="post">
  <select name="x">
    <option value="...">...</option>
  </select>
</form>
```

13 HTTP / MYSQL RESULTS

Request flow Browser request → web server → PHP interpreter → MySQL. The result returns to PHP, which creates HTML/text for the browser.

Query result `mysqli_query()` returns a result handle/object, not a PHP row array. Fetch rows with `mysqli_fetch_all($r, MYSQLI_ASSOC)`.

Associative arrays `key` ⇒ `value`; with `MYSQLI_ASSOC`, database column names become keys, e.g. `$row["ID"]`.

GET / POST `$_GET` comes from URL query parameters; `$_POST` comes from the request body. `$_SERVER["REQUEST_METHOD"]` identifies the method received by the page.

COND Use POST for form-body submissions; reading a variable from `$_GET`/`$_POST` is separate from validating its value.